Transcript Unavailable

00:01

अगर अभी भी कोई हिस्सा उलझन में लग रहा हो, तो बताना—हम और भी उदाहरण या अलग दृष्टिकोण से समझाने की कोशिश करेंगे। कभी-कभी बस थोड़ा अलग उदाहरण या एक और तरीके से समझना सारी चीज़ें साफ कर देता है!

i dont understand'

No problem 👍 Let's forget the optimized code for a moment and understand the **idea slowly with a simple example**.

We want:

$$(arr[i] + arr[j] + arr[k])\%d = 0$$

Example:

```
arr = [3, 3, 4, 7, 8]
d = 5
```

Step 1 — Understand the goal

We must find **3 numbers whose sum is divisible by 5**.

Example:

```
3 + 3 + 4 = 10
10 % 5 = 0 ✅
```

So this triplet works.

Step 2 — Fix the first number (i)

Take the first element.

```
i = 0
arr[i] = 3
```

Now we will try all numbers after it.

Step 3 — Fix the second number (j)

Take the next element.

```
j = 1
arr[j] = 3
```

Current sum:

```
3 + 3 = 6
```

Now we need a **third number k** such that:

```
(6 + k) % 5 = 0
```

Step 4 — Find what k should be

First find remainder of current sum:

```
6 % 5 = 1
```

To make it divisible by 5 we need:

```
1 + ? = 5
```

So the required remainder is:

```
4
```

Meaning:

```
k % 5 = 4
```

So the third number must give remainder **4 when divided by 5**.

Step 5 — Check array

Array:

```
[3, 3, 4, 7, 8]
```

Remainders:

```
3 % 5 = 3
3 % 5 = 3
4 % 5 = 4
7 % 5 = 2
8 % 5 = 3
```

We needed remainder **4**.

We have:

```
4
```

So triplet:

```
3, 3, 4
```

works.

Step 6 — Why we use freq array

Instead of searching every time for `k`, we **store remainders we already saw**.

Example freq array for `d = 5`:

```
index = remainder
freq[0]
freq[1]
freq[2]
```

```
freq[3]
freq[4]
```

If we need remainder **4**, we simply check:

```
freq[4]
```

If it is **1**, we found a triplet.

Step 7 — What this line means

```
required = (d - ((arr[i] + arr[j]) % d)) % d;
```

It simply calculates:

```
what remainder the third number must have
```

Example:

```
arr[i] + arr[j] = 6
6 % 5 = 1
```

```
required = 5 - 1 = 4
```

So the third number must have remainder **4**.

Step 8 — What this line means

```
count += freq[required];
```

If we already saw numbers with remainder **4**, we add them to answer.

Step 9 — What this line means

```
freq[mod]++;
```

We store remainder of current number for future triplets.

✅ **In simple words**

Algorithm does:

1️⃣ Fix first number
2️⃣ Fix second number
3️⃣ Calculate what remainder the third number must have
4️⃣ Check if we already saw such numbers

💡 If you want, I can also show **a very simple visual diagram of this algorithm**.
Most developers understand it in **2 minutes after seeing that diagram.**

---

required = (d - ((arr[i] + arr[j]) % d)) % d; what is last % for ?? as u said arr[i] + arr[j] = 6
6 % 5 = 1 mean ((arr[i] + arr[j]) % d)) this part 5 - 1 mean d - ((arr[i] + arr[j]) % d)) .

Good question 👍 The **last** `% d` is used to handle a **special case when the remainder is already 0**. Let's go step by step.

The formula is:

Java

Step 1 — Normal Case (like your example)

Example:

```
arr[i] + arr[j] = 6
d = 5
```

First part:

```
(arr[i] + arr[j]) % d
6 % 5 = 1
```

Second part:

```
d - 1 = 4
```

Last `% d`:

```
4 % 5 = 4
```

So

```
required = 4
```

Meaning:

```
k % 5 = 4
```

So the third number must give remainder **4**.

Step 2 — Why the last `% d` is needed

Consider another example.

```
arr[i] = 3
arr[j] = 7
```

```
3 + 7 = 10
10 % 5 = 0
```

Now apply formula **without last** `%`:

```
d - 0 = 5
```

This means:

```
required = 5
```

But this is **wrong** because remainder values are only:

```
0,1,2,3,4
```

There is **no remainder 5**.

Step 3 — Now apply the last `% d`

```
required = (5 - 0) % 5
required = 5 % 5
required = 0
```

Now correct ✅

Meaning:

```
k % 5 = 0
```

So the third number must be divisible by 5.

Step 4 — Simple rule

The last `% d` ensures:

```
required remainder is always between 0 and d-1
```

Final Intuition

We are solving this equation:

```
(arr[i] + arr[j] + arr[k]) % d = 0
```

Let

```
x = (arr[i] + arr[j]) % d
```

Then we need:

```
(x + arr[k]) % d = 0
```

So

```
arr[k] % d = (d - x) % d
```

That is why the formula is:

```
required = (d - ((arr[i] + arr[j]) % d)) % d;
```

✅ Short interview explanation

The last `% d` handles the case when `(arr[i] + arr[j]) % d = 0`, ensuring the required remainder stays in the valid range `[0, d-1]`.

If you want, I can also show a **much easier formula used by many engineers**:

```
required = (d - sumMod) % d
```

and a **visual trick that makes this whole problem obvious in 30 seconds.**

i dont understand this two line count += freq[required];
freq[mod]++;

Good — you are asking the **most important part of the code**. Let's understand these **two lines very slowly with a small example**.

The two lines:

```
count += freq[required];
```

Java

First understand what `freq` means

Java

`freq` stores **how many numbers we already saw with a certain remainder**.

Example:

If `d = 5`

```
index (remainder): 0 1 2 3 4
freq value      : 0 0 0 0 0
```

Meaning:

```
freq[3] = 2
```

means **we already saw 2 numbers whose remainder is 3**.

Example Walkthrough

Array:

```
arr = [3,3,4,7,8]
d = 5
```

Assume:

```
i = 0 → arr[i] = 3
```

Now we start checking `j`.

Step 1

```
j = 1 → arr[j] = 3
```

Find remainder:

```
mod = 3 % 5 = 3
```

Calculate required remainder for third number:

```
3 + 3 = 6
6 % 5 = 1
required = 5 - 1 = 4
```

So we need:

```
k % 5 = 4
```

Line 1

‹/› Java

```
freq[4] = 0
```

Meaning **we haven't seen any number with remainder 4 yet**.

So:

```
count += 0
```

Triplets found:

```
0
```

Line 2

‹/› Java

```
mod = 3
```

So we store that we saw a number with remainder 3.

Now:

```
freq = [0,0,0,1,0]
```

Meaning **one number with remainder 3 exists**.

Step 2

Now

```
j = 2 → arr[j] = 4
```

```
mod = 4
```

Find required:

```
3 + 4 = 7
7 % 5 = 2
required = 5 - 2 = 3
```

So we need:

```
k % 5 = 3
```

Line 1 again

```Java
freq[3] = 1
```

That means **we already saw one number with remainder 3**.

So we found **one valid triplet**.

```
count = 1
```

Triplet is:

```
3,3,4
```

Line 2 again

```Java
mod = 4
```

Update freq:

```
freq = [0,0,0,1,1]
```

Now we know **one number with remainder 4 exists**.

Very Simple Meaning of the Two Lines

Line 1

```Java
```

👉 "How many numbers already exist that can complete the triplet?"

Line 2

```Java
```

👉 "Store the current number's remainder so future elements can use it."

Super Simple Analogy

Imagine `freq` is a **basket of remainders we've seen**.

When a new number comes:

1️⃣ Check if the basket already has the remainder you need → add to `count`.

2️⃣ Put the current remainder into the basket.

✅ One-line summary

```
count += freq[required]; → find matching previous numbers
freq[mod]++; → store current number for future matches
```

If you want, I can also show you **a MUCH simpler way to solve this problem (most senior engineers use it)** which removes this confusing logic completely.